TUNE — Technical Architecture for High-End Audio Integration

Prepared for Vincent Briet, Totaldac — April 2026

About — MozAIk Labs

Bertrand Clech — Founder & Lead Developer

EPFL Engineer (Communication Systems — IT/Telecom convergence, 1995). 30+ years of experience in software engineering, telecom, and distributed systems. Audiophile and entrepreneur, passionate about bridging the gap between high-end audio and modern software.

MozAIk Labs is a French company focused on building the next generation of music server software for audiophiles. Our mission: deliver studio-quality playback with the convenience of modern streaming — without compromise.

Founder	Bertrand Clech
Company	MozAIk Labs
Website	mozaiklabs.fr
Location	France
Product	Tune — Multi-room Music Server
Status	Beta (v0.5.2, April 2026)
Beta testers	Active community via mozaiklabs.fr/forum

Audio Equipment:

- Micromega M-One (amplifier/DAC/streamer)
- EverSolo DMP-A8 (streamer)
- Lindemann (streamer)
- Sonos (multi-room)

Streaming Services: Tidal HiFi+, Qobuz Studio

Team:

- Bertrand — Architecture, backend (Python/FastAPI), iOS (Swift/SwiftUI), infrastructure
- Matteo — Frontend, ecommerce (React/Laravel)
- JP — Architecture advisor
- Freddy — HiFi hardware partnerships (Belgium)
- Claude AI (Anthropic) — AI-assisted development & rapid prototyping

---

1. What is Tune?

A **multi-room music server** that unifies local libraries, network shares, and 6 streaming services (Tidal, Qobuz, Spotify, YouTube, Deezer, Amazon Music) with **bit-perfect playback** to DLNA/UPnP renderers,

AirPlay devices, and local outputs.

Key differentiators:

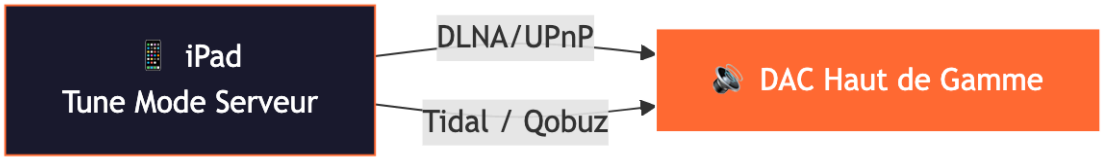
- Bit-perfect and native DSD passthrough
- Federated search across all sources
- Multi-room with synchronization
- Open architecture, self-hosted
- Runs on Linux, macOS, Windows, iPadOS, iOS, Android

---

2. Technology Stack by Platform

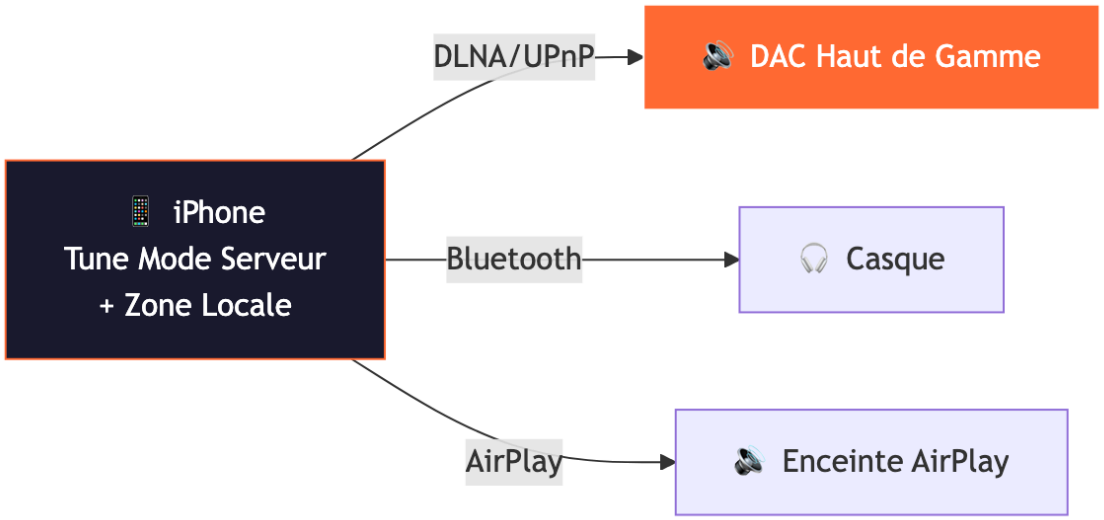
1b. Use Cases — How Tune Fits Your Setup

Scenario 1: iPad only (standalone)



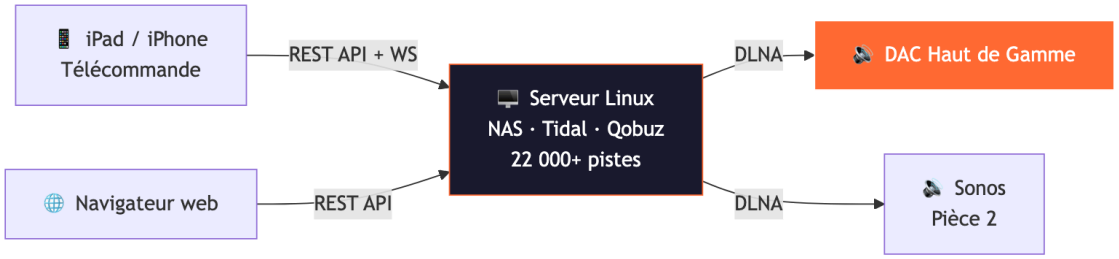
- iPad runs Tune in **server mode** (embedded engine)
- Scans local music (iPad storage + Apple Music library)
- Connects to streaming services (Tidal, Qobuz)
- Sends audio via **DLNA/UPnP** directly to your DAC
- Controls playback from the iPad touchscreen
- **Multi-room capable**: iPad discovers multiple DLNA renderers, can group zones for synchronized playback
- **Best for**: simple setup, one or multiple rooms, audiophile on the go

Scenario 1b: iPhone only (portable audiophile)



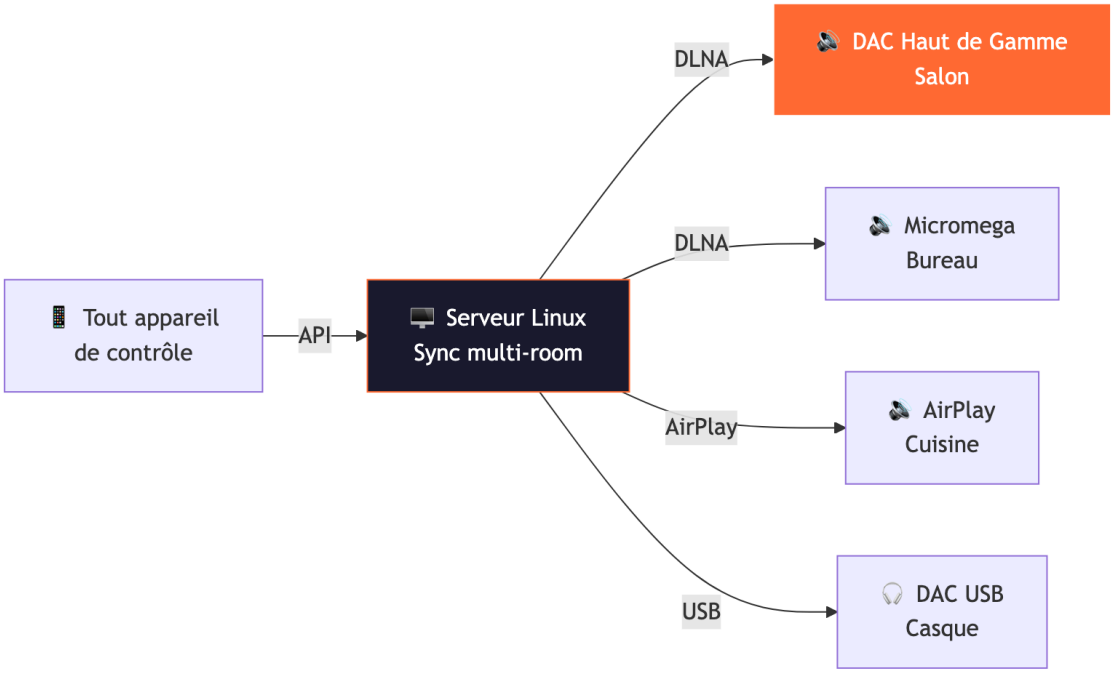
- iPhone runs Tune in **server mode** (autonomous, no external server needed)
- Local zone streams from Tidal/Qobuz via the device
- Can output to **DLNA renderers on the same network**, Bluetooth headphones, or AirPlay
- Full control interface with library, search, playlists, favorites
- **Best for**: portable listening, traveling audiophile, quick DLNA control

Scenario 2: Linux server + iPad/iPhone remote



- Linux server (Intel NUC, Raspberry Pi, or any PC) runs **tune-server**
- Scans NAS/SMB shares, local folders, streaming services
- Full library with PostgreSQL, metadata enrichment, playlists
- iPad/iPhone/Mac connects as **remote control** via WiFi (REST API + WebSocket)
- Web browser access at `http://server:8888` for any device
- Server sends audio to **multiple DLNA renderers** simultaneously (multi-room)
- **Best for**: serious audiophile setup, large library, multi-room, NAS storage

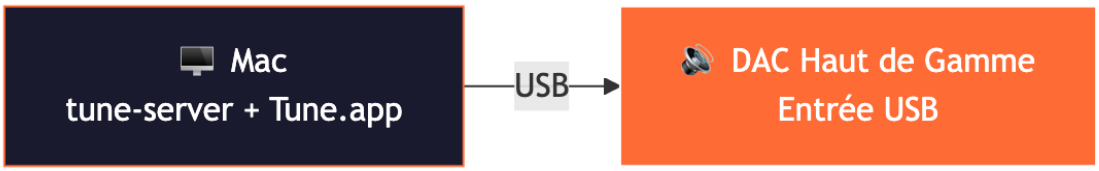
Scenario 3: Linux server + multiple outputs



- Same Linux server drives **multiple zones simultaneously**
- Each zone has its own queue, volume, and output
- Zones can be **grouped for synchronized playback** (multi-room)
- Per-zone sync delay compensation for network latency

- Mix of DLNA, AirPlay, and USB DAC outputs
- **Best for**: whole-house audio, mixed equipment, audiophile + casual rooms

Scenario 4: Mac desktop (all-in-one)

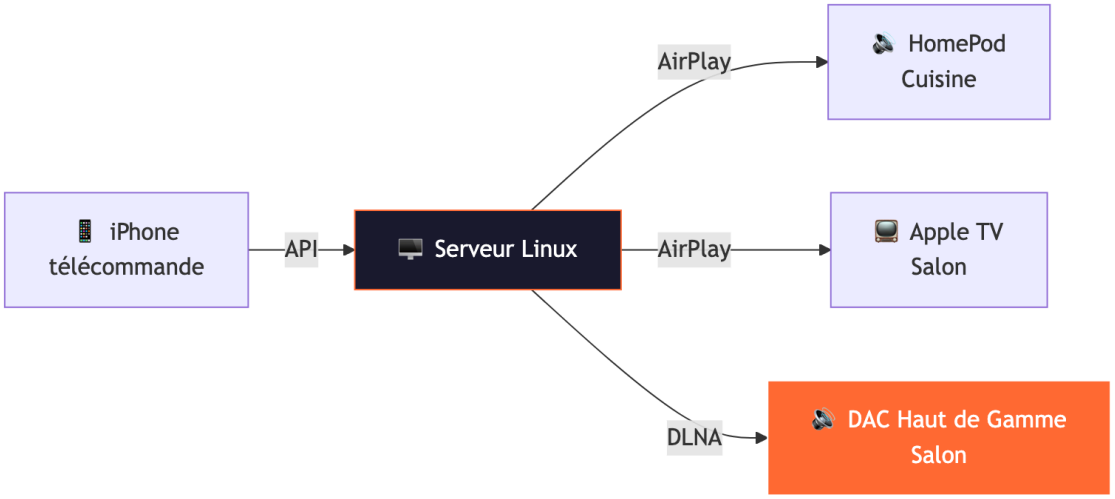


- Mac runs both **tune-server** (terminal) and **Tune.app** (native UI)
- Direct USB output to DAC (sounddevice, exclusive mode planned)
- Web UI accessible from any browser on the network
- **Best for**: desktop audiophile, headphone setup, simple one-box solution

Scenario 5: Raspberry Pi (embedded audiophile)

- Dedicated Raspberry Pi runs tune-server headless (no screen)
- Music on USB SSD or mounted NAS
- USB output to DAC (bit-perfect, exclusive mode)
- Control from any phone/tablet/browser on the network
- **Best for**: ultra-low-cost dedicated streamer, minimalist audiophile

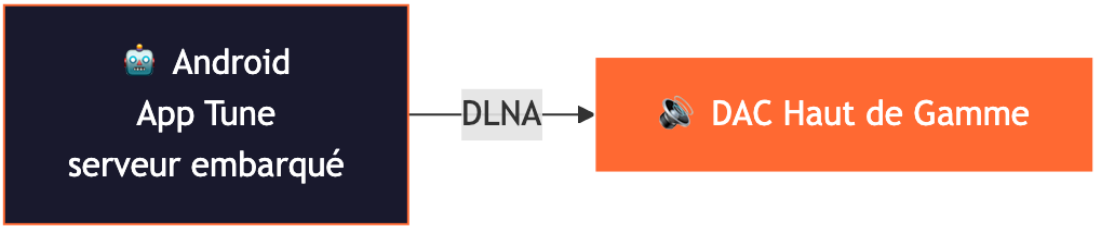
Scenario 6: iPhone as remote + AirPlay



- iPhone controls the server (native app or web)
- Server streams to **AirPlay and DLNA simultaneously**
- Mix Apple ecosystem + audiophile DLNA renderers

- **Best for**: Apple household with high-end audio in one room

Scenario 7: Android phone + Flutter app

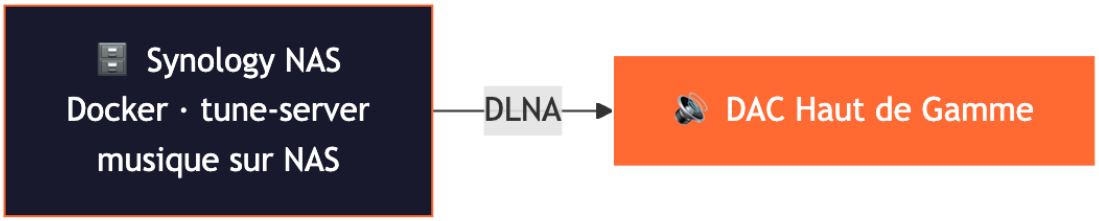


- Android runs Tune with embedded server (Flutter)
- Direct DLNA output to DAC
- Streaming services (Tidal, Qobuz)
- **Best for**: Android users, portable setup

Scenario 8: Windows PC (office/studio)

- Windows runs tune-server as tray application
- USB or DLNA output
- Web UI from any browser
- **Best for**: studio/office, Windows-only environments

Scenario 9: Docker (NAS / homelab)



- Runs in Docker on NAS (Synology, QNAP, Unraid)
- Music library already on the NAS — zero copy
- DLNA output to any renderer on the network
- **Best for**: existing NAS owners, zero-hardware solution

Quick Comparison

Setup	Hardware	Library	Multi-room	Audio Path	Complexity

iPad only	iPad	Local + streaming	Yes (DLNA)	iPad → DLNA → DAC(s)	★☆☆
Linux + remote	Server + iPad	NAS + streaming	Yes	Server → DLNA → DAC	★★☆
Linux + multi	Server + any	NAS + streaming	Yes (sync)	Server → multiple outputs	★★★
Mac desktop	Mac	Local + streaming	Optional	Mac → USB → DAC	★☆☆
Raspberry Pi	RPi + SSD	SSD/NAS + streaming	Optional	RPi → USB → DAC	★★☆
iPhone + AirPlay	Server + iPhone	NAS + streaming	Yes	Server → AirPlay/DLNA	★★☆
Android	Android phone	Local + streaming	No	Phone → DLNA → DAC	★☆☆
Windows	PC	Local + streaming	Optional	PC → USB/DLNA → DAC	★☆☆
Docker / NAS	NAS	NAS volumes	Yes	NAS → DLNA → DAC	★★☆

Linux / macOS / Windows Server

Layer	Technology	Purpose
Language	Python 3.11+ (async)	Server core
API	FastAPI + Uvicorn	106+ REST endpoints + WebSocket
Database	SQLite / **PostgreSQL** (dual engine)	Library, playlists, zones
Audio Pipeline	FFmpeg	Decode, transcode, resample
DLNA/UPnP	async-upnp-client	Renderer control, SSDP discovery
AirPlay	pyatv	Apple device streaming
Local Output	sounddevice + numpy	USB DAC / soundcard
HTTP Streamer	aiohttp (:8080)	Audio serving for DLNA renderers
Metadata	mutagen + musicbrainzngs	Tag reading/writing, enrichment
File Watching	watchfiles	Real-time library refresh

iPadOS / iOS / macOS (SwiftUI)

Layer	Technology	Purpose

Language	Swift 6.0 (strict concurrency)	Native app
UI	SwiftUI	Responsive iPad/iPhone/Mac
Database	GRDB 7.0+	SQLite ORM
Audio	AVPlayer	All formats via HTTP streaming
DLNA	Native XMLParser + URLSession	UPnP control
Streaming	Native Swift (no dependencies)	Tidal, Qobuz, YouTube

Flutter (Android / iOS)

Layer	Technology	Purpose
Language	Dart 3.11+	Cross-platform
Database	Drift 2.20+	SQLite ORM
Audio	just_audio	Native platform decoders
HTTP Server	Shelf + shelf_router	Embedded REST API

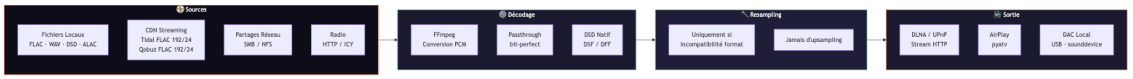
Web Client

Layer	Technology	Purpose
Language	TypeScript 5.7+	Type-safe SPA
Framework	Svelte 5 (runes)	Reactive UI
Build	Vite 6.0+	Fast build
Design	3 breakpoints	Desktop / Tablet / Mobile

---

3. Audio Architecture

Signal Path



Playback Strategies

Strategy	When	CPU	Quality
Direct URL Passthrough	Streaming → DLNA	Zero	Bit-perfect
Native DSD Passthrough	DSF/DFF → DSD-capable renderer	Zero	Bit-perfect
File Passthrough	Local FLAC → FLAC-capable renderer	Minimal	Bit-perfect
FFmpeg Transcode	Format mismatch	Medium	Transparent

Supported Formats & Quality

Format	Max Resolution	DSD	Gapless
FLAC	192 kHz / 24-bit	—	✓
WAV	192 kHz / 32-bit	—	✓
ALAC	192 kHz / 24-bit	—	✓
DSD (DSF/DFF)	DSD128 (5.6 MHz)	Native	✓
DSD Fallback	176.4 kHz / 24-bit PCM	Converted	✓
AAC/MP3/OGG	48 kHz / 16-bit	—	✓

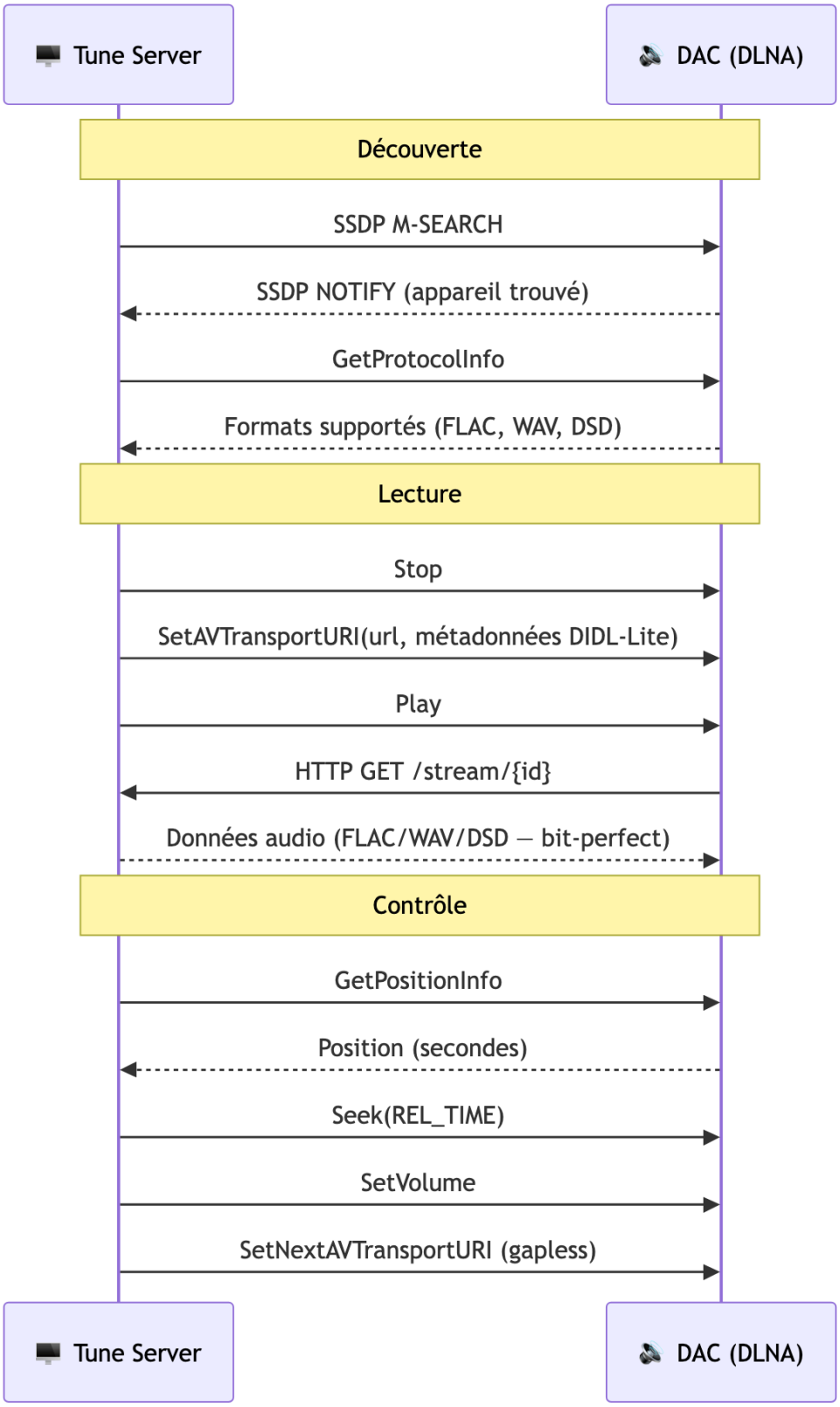
Streaming Service Quality

Service	Max Quality	Format	Resolution
Tidal	HI_RES_LOSSLESS	FLAC	192 kHz / 24-bit
Qobuz	Studio Ultra	FLAC	192 kHz / 24-bit
Amazon Music	ULTRA_HD	FLAC	96 kHz / 24-bit
Deezer	HiFi	FLAC	44.1 kHz / 16-bit
Spotify	Premium	OGG 320k	Lossy
YouTube	Best available	AAC/OPUS	Variable

---

4. DLNA/UPnP Implementation Details

Renderer Communication



DIDL-Lite Metadata

Every track sent to the renderer includes full metadata:

```
<DIDL-Lite>
  <item>
    <dc:title>Track Title</dc:title>
    <upnp:artist>Artist Name</upnp:artist>
    <upnp:album>Album Title</upnp:album>
    <res protocolInfo="http-get:*:audio/flac:*"
         sampleFrequency="96000"
         bitsPerSample="24"
         nrAudioChannels="2"
         duration="0:04:32.000">
      http://192.168.1.29:8080/stream/abc123
    </res>
    <upnp:albumArtURI>http://192.168.1.29:8080/artwork/cover.jpg</upnp:albumArtURI>
  </item>
</DIDL-Lite>
```

DSD Detection

```
# Auto-detect DSD capability from GetProtocolInfo or device name
if "audio/x-dsf" in sink_protocols or "DSD" in device_model:
    # Serve DSF/DFF files bit-perfect
    # MIME: audio/x-dsf or audio/x-dff
else:
    # Transcode to 176.4kHz/24-bit PCM via FFmpeg
```

---

5. Multi-Room Architecture

Zone Model

```
Zone 1: "Salon" (DLNA → Totaldac d1-twelve)
  ├─ Queue: [Track A, Track B, Track C]
  ├─ Volume: 0.65
  └─ State: Playing (position: 2:34)

Zone 2: "Bureau" (DLNA → Micromega M-One)
  ├─ Queue: [Track X, Track Y]
  ├─ Volume: 0.40
  └─ State: Paused

Group "Whole House" (leader: Zone 1)
  ├─ Zone 1 (leader, sync_delay: 0ms)
  └─ Zone 2 (follower, sync_delay: +150ms)
```

Synchronization Engine

- Adaptive polling: 1s when playing, 10s when idle
- Drift detection threshold: 500ms

- Per-zone sync delay compensation
- DLNA latency measurement and caching
- Staggered start for network renderers

---

6. Audio Excellence Features

6.1 Clock Synchronization

Current state: Audio clocking is driven by the renderer's internal clock. Tune sends data via HTTP; the renderer buffers and plays at its own clock rate.

A. External Word Clock Support — *Planned*

- Support for external word clock input/output via DLNA renderer
- `ClockSource` negotiation in UPnP GetProtocolInfo

B. Precision Timing Headers — ✅ Implemented (v0.5.2)

- Custom HTTP headers on every audio stream for sample-accurate timing:

```
X-Tune-SampleRate: 192000
X-Tune-BitDepth: 24
X-Tune-Timestamp: 1712345678.123456789 (nanosecond precision)
X-Tune-BitPerfect: true
X-Tune-Format: flac
X-Tune-Channels: 2
```

- Renderer can use these for jitter-free reconstruction

C. RAVENNA/AES67 Output — *Planned*

- PTP (IEEE 1588) clock synchronization — sub-microsecond precision
- Professional-grade network audio transport
- Direct integration with Totaldac's network input

D. Roon-style RAAT Protocol — *Planned*

- Custom audio transport with clock recovery, adaptive buffering, jitter elimination

6.2 Bit-Perfect Verification — ✅ Implemented (v0.5.2)

- ✅ End-to-end MD5 checksum verification (source file → renderer input)
- ✅ Audio hash comparison: `source_hash == output_hash` verified on passthrough
- ✅ Visual indicator in UI: green "Bit-Perfect ✓" or yellow "Transcoded ⚠️"
- ✅ Complete pipeline decision log (every decision recorded and displayed)

6.3 Advanced Resampling — ✅ Implemented (v0.5.2)

- ✅ **Resample policy**: `auto` (default), `never` (refuse incompatible formats), `integer_ratio` (prefer 44.1→88.2→176.4 kHz)
- ✅ User-configurable via Settings UI
- ✅ Integer ratio resampling preference for audiophile-grade conversion
- ✅ SoX resampler via FFmpeg for transparent fallback

6.4 Buffer Management — ✅ Implemented (v0.5.2)

- ✅ Configurable pre-buffer size per output (`TUNE_AUDIO_BUFFER_KB`)
- ✅ Pre-buffer duration before playback start (`TUNE_PREBUFFER_SECONDS`)

- ✅ Low-latency mode (10ms) for local USB DAC
- ✅ High-buffer mode (200ms) for network-challenged environments

6.5 USB Audio Class Support — ✅ Implemented (v0.5.2)

- ✅ **Exclusive mode**: bypass OS mixer (PulseAudio/PipeWire) for bit-perfect USB DAC output
- ✅ Configurable latency: 10ms (ultra-low) to 200ms (safe)
- ✅ Software volume disabled in exclusive mode (pure bit-perfect chain)
- ✅ Native DSD passthrough to DSD-capable renderers
- ✅ Bit-perfect verification via checksum

6.6 Room Correction / DSP — ✅ Implemented (v0.5.2)

- ✅ **DSP filter chain**: any FFmpeg `-af` filter (equalizer, bass, treble, compressor, etc.)
- ✅ **Convolution**: import impulse response files (Dirac Live, REW, Audiolens) for room correction
- ✅ DSP toggle on/off in Settings UI
- ✅ Bypass mode for purists (DSP disabled by default — zero processing)

6.7 Signal Path Display — ✅ Implemented (v0.5.2)

- ✅ **Roon-inspired modal** showing complete signal path:

```
Source: Qobuz FLAC 96/24
→ Transport: Direct URL Passthrough
→ Renderer: Totaldac d1-twelve (DLNA)
→ Clock: Internal (renderer)
→ Processing: None (bit-perfect)
→ Output: 96kHz / 24-bit / 2ch
```

- ✅ Color-coded dot in transport bar (green = bit-perfect, yellow = transcoded)
- ✅ Expandable pipeline decisions log
- ✅ Checksum verification badge
- ✅ Full FR/EN internationalization

---

7. Totaldac Integration Proposal

Phase 1: Basic DLNA (Already Working)

- Tune discovers Totaldac via SSDP
- FLAC/WAV/DSD playback via standard UPnP
- Volume control via SetVolume
- Metadata display on device

Phase 2: Enhanced Integration

- Custom device profile for Totaldac (optimal settings)
- DSD native detection and passthrough
- Gapless via SetNextAVTransportURI
- Signal path reporting

Phase 3: Totaldac-Native Protocol

- Direct TCP audio transport (bypass HTTP overhead)
- PTP clock synchronization
- Native DSD (not DoP)
- Remote control API for Totaldac hardware

- Totaldac-branded Tune interface

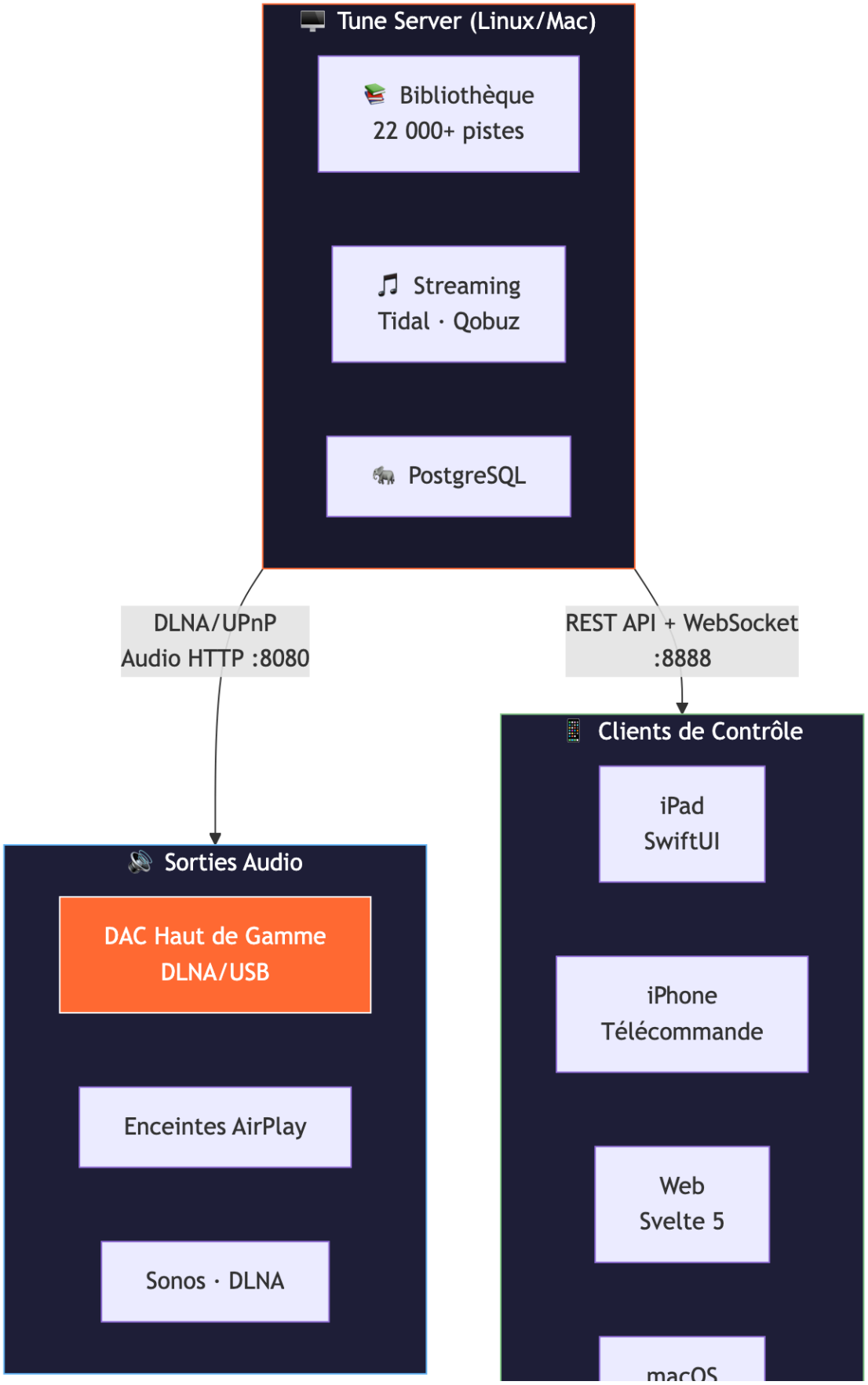
Phase 4: Joint Product

- Tune Server embedded in Totaldac hardware
- Pre-configured Linux image
- Hardware-accelerated audio pipeline
- Word clock integration
- Totaldac d1-twelve with Tune built-in

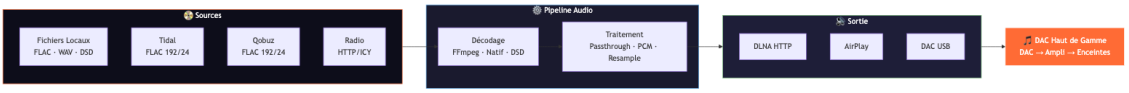
---

8. Architecture Diagrams

Network Topology

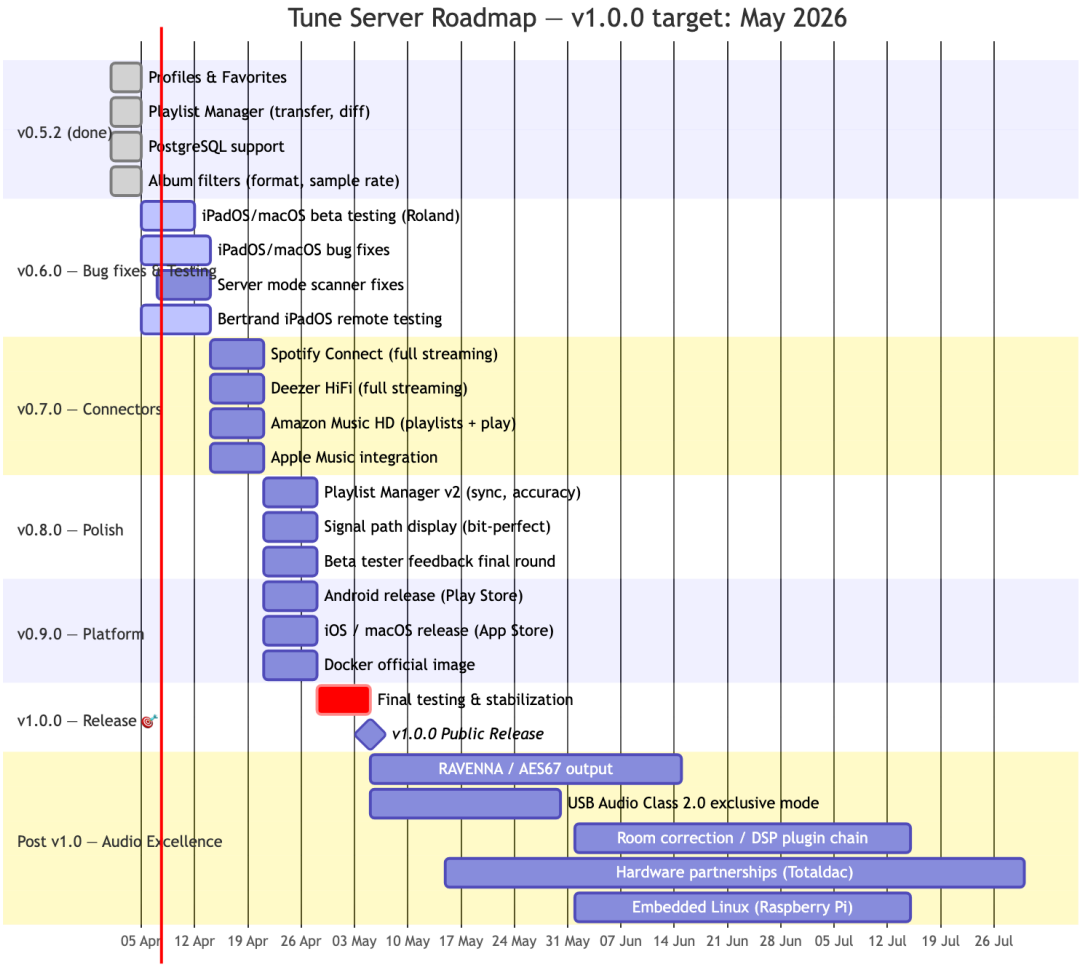

Tune Server (Linux/Mac)
Bibliothèque
22 000+ pistes
Streaming
Tidal · Qobuz
PostgreSQL
DLNA/UPnP
Audio HTTP :8080
REST API + WebSocket
:8888
Sorties Audio
DAC Haut de Gamme
DLNA/USB
Enceintes AirPlay
Sonos · DLNA
Clients de Contrôle
iPad
SwiftUI
iPhone
Télécommande
Web
Svelte 5
macOS

Audio Signal Path



---

9. Roadmap



Current Status (v0.5.2 — April 2026)

Feature	Status	Target

Library management	✅ Production	—
Tidal HiFi+	✅ Full streaming (FLAC 192/24)	—
Qobuz Studio	✅ Full streaming (FLAC 192/24)	—
YouTube Music	✅ Streaming	—
Spotify	⚠️ Preview only	v0.6.0
Deezer	⚠️ Preview only	v0.6.0
Amazon Music	⚠️ Search only	v0.6.0
Apple Music	🔜 iPadOS only	v0.6.0
DLNA/UPnP output	✅ Bit-perfect, DSD native, gapless	—
AirPlay output	✅ Production	—
Multi-room	✅ Zone grouping + sync engine	—
Playlist Manager	✅ Transfer, diff, recovery	—
User Profiles & Favorites	✅ Multi-user	—
Web client	✅ Responsive, 8 languages	—
iPadOS / iOS / macOS	✅ TestFlight	App Store v1.0
Android (Flutter)	🔜 Beta	Play Store v1.0
RAVENNA / AES67	🔜 Planned	Post v1.0
DSP / Room correction	🔜 Planned	Post v1.0
Hardware integration	🔜 Planned	Post v1.0

Path to v1.0.0 (May 2026)

1. **Beta testing & bug fixes** — Active iPadOS/macOS testing with Roland (Germany) and Bertrand, fix scanner, playback, and UI issues
2. **Complete all streaming connectors** — Spotify Connect, Deezer HiFi, Amazon Music HD, Apple Music
3. **Finalize Playlist Manager** — Improve matching accuracy, two-way sync
4. **App Store / Play Store releases** — iOS, macOS, Android public releases
5. **v1.0.0** — Stable, feature-complete, ready for hardware partnerships

---

10. Contact & Resources

- **Website**: https://mozaiklabs.fr
- **Downloads**: https://mozaiklabs.fr/download
- **GitHub**: github.com/renesenses/tune-server-linux
- **Beta Forum**: https://mozaiklabs.fr/forum
- **Current Version**: v0.5.2 (April 2026)

Document prepared by Bertrand & Claude — MozAIk Labs, April 2026